

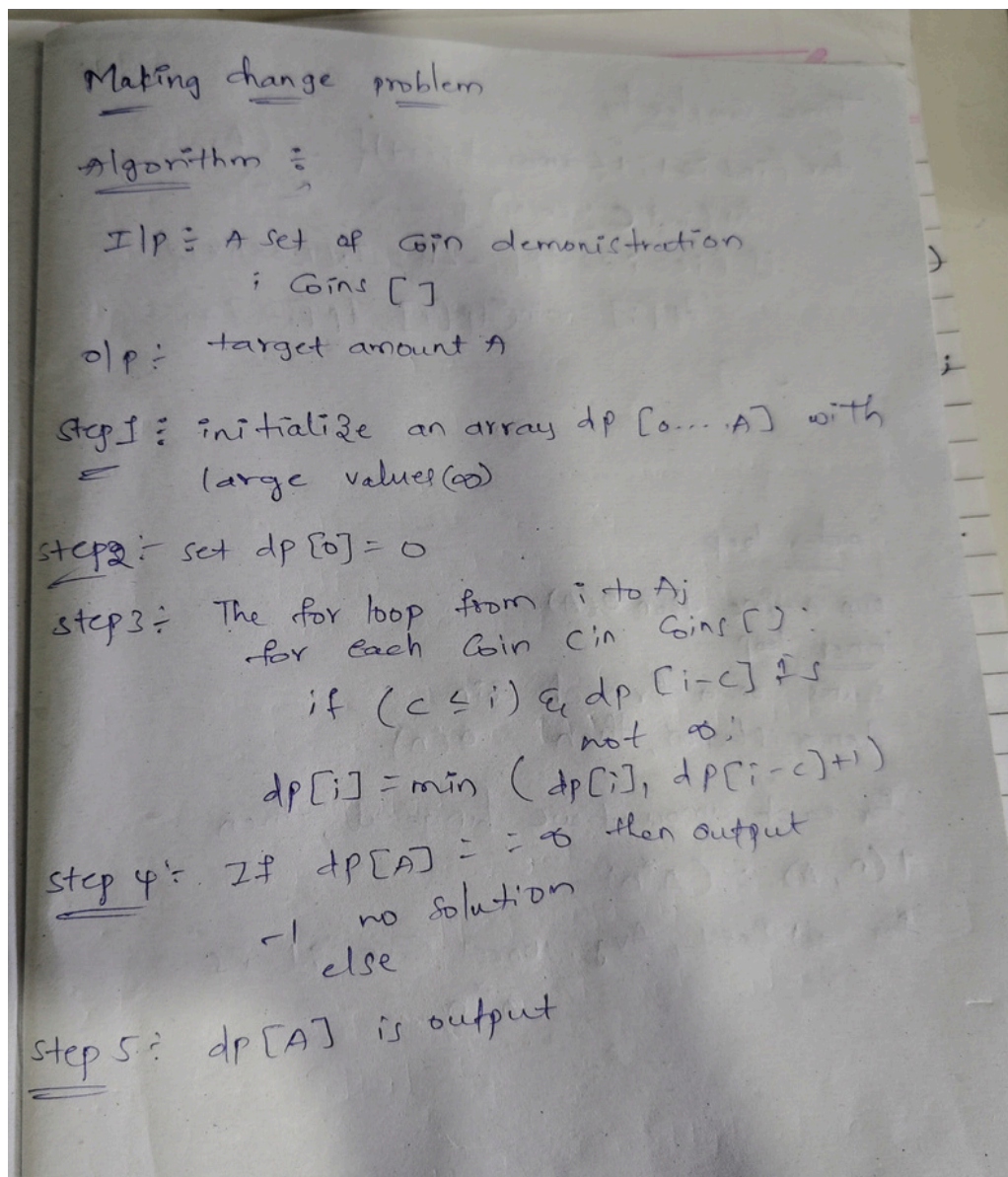
Practical: 7

Implementation of making a change problem using dynamic programming

20/09/2026

Aim: Implementation of making a change problem using dynamic programming

Algorithm:



Making change problem

Algorithm :

I/p : A set of coin denomination
: Coins []

o/p : target amount A

Step 1 : initialize an array dp [0... A] with
= large values(∞)

Step 2 : set dp[0] = 0

Step 3 : The for loop from 1 to A
for each coin c in Coins []:
if ($c \leq i$) & dp[i-c] is not ∞ :
dp[i] = min (dp[i], dp[i-c] + 1)

Step 4 : If dp[A] == ∞ then output
-1 no solution
else

Step 5 : dp[A] is output

Program:

```
#include <bits/stdc++.h>

using namespace std;

//Function to compute minimum coins required
int coinChange(vector<int>&coins, int amount) {
    //DP array to store minimum coins for each value
    vector<int> dp(amount+1, INT_MAX);
    dp[0]=0; //base case: 0 coins needed for amount 0

    // Fill DP table
    for(int i=1; i<=amount; i++){
        for(int c : coins) {
            if(c<=i && dp[i-c]!=INT_MAX) {
                dp[i]=min(dp[i], dp[i-c] + 1);
            }
        }
    }

    return(dp[amount]==INT_MAX) ? -1 : dp[amount];
}

// Example usage
int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;

    int result = coinChange(coins, amount);
    if (result != -1)
        cout<<"Minimum coins required: " << result << endl;
```

else

cout << "No solution possible" << endl;

return 0;

}

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ y.cpp -o y ; if ($?) { .y }
Minimum coins required: 2
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:

Time Complexity

```
for (int i=1; i<=amount; i++) { → (A)
    for (int c: coins) { → O(n)
        if (i<=i && dp[i-c] != INT_MAX) {
            dp[i] = min(dp[i], dp[i-c] + 1);
        }
    }
}
```

outer loop
Run A time then $O(A)$

inner loop
Runs on a for each coin demonstration, then $O(n)$

for each amount we change all, then

$T(n, A) = (A * n)$

for all Best, Avg, worst case it is $A * n$

Conclusion:

Matrix Chain Multiplication and Coin Change both highlight how dynamic programming transforms exponential recursive solutions into efficient polynomial-time algorithms. In conclusion, DP ensures optimal subproblem reuse, yielding uniform complexity across best, average, and worst cases.